

Abstract

Most factories and organizations already run **multi-platform infrastructure** that works — edge devices, GPU servers, Windows/macOS PCs, industrial gateways. The hard part is the *cost of putting AI on top of it*: keying every node, exfiltrating data to the cloud, and rewriting integrations is heavy and risky.

DureClaw does not replace any of it. It adds **one real-time collaboration bus** on top of infrastructure that already runs, and lets scattered machines join in a single line — **keyless**. A **master brain** (Claude) absorbs inference, and each node becomes a *different kind of hands* (physical actuators, the browser, the desktop GUI). Human-approved decisions are **compiled into deterministic rules** and replayed with **zero LLM calls**. It is the *starting point* — from data collection and analysis, to automation, to **AX (AI Transformation)**.

1. The Problem

Four recurring barriers when bringing AI to scattered infrastructure:

Barrier	Description
Keys & cost	Per-node model keys explode secret-sprawl, cost, and rate limits.
Data sovereignty	The moment raw telemetry, video, or audio leaves the floor, it becomes a regulatory and security liability.
Integration	Wiring heterogeneous OS/CPU/tools (Pi, GPU, Win, Mac, PLC) becomes a one-off integration project every time.
Trust	When an LLM acts directly, the result is non-deterministic and unauditable.

The guiding principle is **data-acquisition-first**: data you don't capture is lost forever. Before a perfect pipeline, start collecting on the infrastructure you already run — the easiest way possible.

2. Core Idea

2.1 One brain, many hands

The master (Claude) is the **brain**; each node is a **different kind of hands**. Over one bus and one keyless delegation, a single line of reasoning reaches into the **physical world, the browser, or the desktop**.

```
master brain —fan-out—┌ edgeclaw — hands in the physical world (shell·sensor·GPIO·LED·buzzer
(Claude · vision)    ├── webclaw  — hands in the browser      (fetch·DOM, CORS-free)
                    ├── deskclaw — hands on the desktop GUI  (screenshot·click·type·launch)
                    └── adapters — hands of existing tools    (pico·nano·zero·null)
```

2.2 Keyless edges

Edge nodes hold **no model key**. When a natural-language task arrives, the node delegates the prompt to the master brain's `/brain/exec` and receives only the result. Cost, keys, and rate limits are absorbed in one place — the master.

2.3 Decision freezing (LLM as compiler)

The LLM is not called every time. A decision a human has **approved once** is **frozen** into a deterministic rule or macro, and the same problem is replayed in microseconds with **zero LLM calls**. The LLM is a *compiler*, not a runtime dependency — this is the heart of the closed learning loop.

3. Architecture

3.1 The collaboration bus

The DureClaw bus is a single WebSocket channel over **Phoenix Channel** (Elixir). The wire protocol is a 5-tuple JSON frame:

```
[join_ref, ref, topic, event, payload]
```

- **Join**: `phx_join` → topic `work:<WORK_KEY>`. Registers role and capabilities in presence.
- **Dispatch**: the master fans out `task.assign` (to a specific node or `broadcast`).
- **Result**: a node pushes `task.result` (must include the connection's `join_ref`).
- **State**: approvals/decisions propagate via `task.result{approved}` or `state.update`.
- **Heartbeat**: `phoenix / heartbeat` keeps presence alive.

Auxiliary REST API: `POST /api/task`, `GET /api/task-result/:id`, `/api/presence`, `/api/work-keys/latest`, `/api/health`. Auth is a Bearer token (`OA_H_SECRET`).

3.2 The master brain

The master collects fan-in from the nodes, reasons, and only **proposes** — it never acts directly. Through the keyless delegation endpoint (`/brain/exec`) it handles edge nodes' natural-language tasks on their behalf. This separates

vision and reasoning at the center from action at the edge.

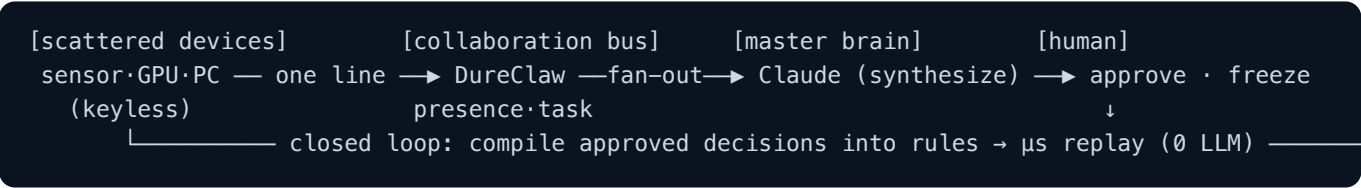
3.3 The node family

Native nodes — built bus-first, one-line join, keyless:

Node	Kind of hands	Implementation
edgeclaw	Physical world — shell·sensor·GPIO·LED·buzzer·relay·signal tower·PA voice	Single static Go binary, no CGo. Win·Mac·Linux·Pi Zero (armv6)·riscv64··· physical-edge runs on any Linux exposing a gpiochip (Pi·Jetson·industrial gateway)
webclaw	The browser — fetch·DOM (CORS-free, always-on)	Chrome MV3 extension, pure JS
deskclaw	Desktop GUI — screenshot·click·type·key·launch + RPA record→replay	Win/macOS, pure Go/no-CGo (built-in OS tooling)

Adapters — bring an existing open-source tool onto the bus with a `durecclaw/` bridge: **picoclawn** (Go) · **nanobot** (Py) · **zeroclawn** (Rust) · **nullclawn** (Zig). Same wire protocol, same keyless delegation.

3.4 Message flow



4. Two concrete forms of freezing

Not an abstract principle — it runs in **two real places**:

- Skill cache on the line.** When the LLM performs a judgment such as a defect investigation on the first pass, the result is cached as a deterministic rule. The same pattern is then applied instantly, with no LLM. (Open MES Korea `EXT-5` integration hub.)
- RPA macros on the desktop.** In deskclaw, when the master (vision) teaches *launch a program → click the right menu → type* one grounded step at a time, deskclaw **records** those steps into a macro (JSON). Afterwards `[RUN] <name>` replays them deterministically with **no LLM**.

Same closed loop, two embodiments — *teach once, then replay forever without the LLM*.

5. Security & Governance

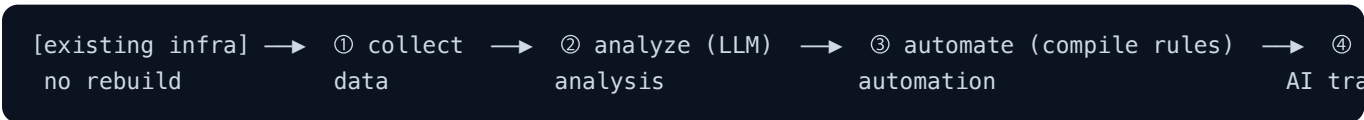
- **Humans decide.** The master only proposes; significant changes require human approval. Approval is the execution trigger.
 - **Keyless edges.** With no model key on the edge, the key-leak surface shrinks.
 - **Data stays at the edge.** Raw telemetry and video never leave the floor; only *refined context* goes to the master.
 - **Auditable.** Every write and approval is recorded in an audit log (Open MES Korea AuditLog). Decision freezing also removes *non-determinism*.
 - **Minimal footprint.** Brokerless HTTP/WS push first — start without a new message queue or agent runtime.
-

6. Use Case — Distributed Edge × Manufacturing MES

- The **dure-factory** demo combines the DureClaw bus with **Open MES Korea** (an AI-native MES). The golden scenario:
1. **Sense** — Pi (`executor@pi-cam`), GPU, and PC nodes join the bus keyless. A defect trigger fires.
 2. **Synthesize** — the master fans in signals from many nodes and reasons about the defect cause by role, then proposes.
 3. **Approve** — a quality manager reviews and approves in the MES UI → `quarantine` .
 4. **Physical result** — when the approval is broadcast over the bus, an edge node fires a  LED, buzzer, and voice. *The AI decision returns to the physical world.*
 5. **Freeze** — the approved judgment is compiled into a skill-cache rule; the next identical case costs zero LLM calls.

A single defect fans out to many machines in about a second, Claude synthesizes a structured suggestion, a human approves, the decision returns to the physical world, and it is frozen into a reusable rule — with an audit trail throughout.

7. From here to AX



DureClaw provides a *continuous path* from ① to ④. Each stage reuses the data and approvals of the previous one, and the closed loop lowers LLM dependence over time.

8. Roadmap

- **deskclaw** — accessibility-tree targeting (pixels → elements, robust to window moves), a screenshot→brain upload loop, macro parameters.
- **edgeclaw** — more actuator profiles (Modbus/OPC-UA relays), industrial-protocol SourceAdapters.
- **bus** — versioning and rollout of frozen decision rules, multi-work-key orchestration.
- **MES** — richer cross-source investigation over time-series (EXT-1) and multimedia (EXT-2), predictive maintenance (EXT-3).

9. Conclusion

DureClaw is not a *new AI platform* but a **thin layer that turns infrastructure you already run into a collaborating AI crew**. Keyless edges, one-brain-many-hands, decision-freezing — with these three principles you can go from data collection to AX *without a rebuild*.

Data at the edge · brains distributed · learning in a closed loop · humans decide.

Appendix A. Wire protocol

Event	Direction	Meaning
phx_join	node → bus	join <code>work:<WK></code> + register presence (role, capabilities)
task.assign	bus → node	dispatch a task (<code>to</code> : node name or <code>broadcast</code>)
task.result	node → bus	return a result (must include the connection's <code>join_ref</code>)
state.update	bus → node	propagate state/decisions (e.g. <code>decision_<lot>: quarantine</code>)
heartbeat	node → bus	keep presence alive (15-30s)

Appendix B. Node environment variables (common)

`STATE_SERVER` (bus host:port) · `OAH_SECRET` (Bearer) · `WORK_KEY` ·
`AGENT_NAME` / `AGENT_ROLE` / `CAPABILITIES` · `BRAIN_URL` / `BRAIN_TOKEN` (keyless LLM delegation).